

Package ‘dwf4r’

September 22, 2026

Type Package

Title Unofficial R Interface to the Digilent WaveForms SDK

Version 0.2.0

Description An independently developed R interface to selected functions of the Digilent WaveForms SDK and its DWF library. The current version supports only a limited subset of the SDK, focused on signal generation and timing control through the Analog Out and Digital Out instruments. The WaveForms SDK must be installed separately.

License MIT + file LICENSE

URL https://mass-spec.ru/projects/device_control/dwf4r/eng/

BugReports <https://github.com/andreysamokhin/dwf4r/issues>

Encoding UTF-8

LazyData true

NeedsCompilation yes

SystemRequirements Digilent WaveForms SDK (installed separately)

Imports checkmate,
Rcpp

LinkingTo Rcpp

Suggests testthat (>= 3.0.0)

Config/testthat/edition 3

RoxygenNote 7.3.3

Author Andrey Samokhin [aut, cre, cph] (ORCID: <<https://orcid.org/0000-0003-0223-6087>>)

Maintainer Andrey Samokhin <andrey.s.samokhin@gmail.com>

VignetteBuilder knitr

Contents

CarrierEnableDisable	3
CloseAllDevices	3
CloseDevice	4
ConfigureAnalogOut	5
ConfigureDigitalOut	5
DigitalOutChannelEnableDisable	6

GetAnalogOutIdle	7
GetAnalogOutInfo	8
GetAnalogOutMaster	9
GetAnalogOutRemaining	10
GetAnalogOutRepeatTrigger	11
GetAnalogOutStatus	11
GetAnalogOutTiming	12
GetAnalogOutTriggerSlope	13
GetAnalogOutTriggerSource	13
GetCarrierFunction	14
GetCarrierSettings	14
GetCarrierValues	15
GetDeviceTriggerInfo	16
GetDigitalOutChannelInfo	17
GetDigitalOutChannelSettings	18
GetDigitalOutCounter	19
GetDigitalOutCustomSampleRate	20
GetDigitalOutDividerValues	21
GetDigitalOutInfo	22
GetDigitalOutInitialCounter	23
GetDigitalOutRemaining	24
GetDigitalOutRepeatTrigger	25
GetDigitalOutStatus	26
GetDigitalOutTiming	26
GetDigitalOutTriggerSlope	28
GetDigitalOutTriggerSource	28
IsCarrierEnabled	29
IsDigitalOutChannelEnabled	29
ListDevices	30
OpenDevice	31
ResetAnalogOut	32
ResetDigitalOut	33
SetAnalogOutIdle	33
SetAnalogOutMaster	34
SetAnalogOutRepeatTrigger	35
SetAnalogOutTiming	36
SetAnalogOutTriggerSlope	37
SetAnalogOutTriggerSource	38
SetCarrier	38
SetCarrierData	40
SetDigitalOutChannelSettings	41
SetDigitalOutCounter	42
SetDigitalOutCustomPattern	43
SetDigitalOutCustomSampleRate	45
SetDigitalOutDivider	46
SetDigitalOutInitialCounter	47
SetDigitalOutInitialDivider	48
SetDigitalOutRepeatTrigger	49
SetDigitalOutTiming	50
SetDigitalOutTriggerSlope	51
SetDigitalOutTriggerSource	52
TriggerDevice	53

Index**54**

CarrierEnableDisable *Enable or disable carrier output*

Description

Enable or disable the carrier node for an Analog Out channel.

EnableCarrier() enables the carrier node.

DisableCarrier() disables the carrier node.

Usage

```
EnableCarrier(device, channel)
```

```
DisableCarrier(device, channel)
```

Arguments

device A "dwf4r_device" object returned by OpenDevice() that refers to a device connection that is still open.

channel An integer scalar representing the zero-based Analog Out channel index.

Details

These functions change the carrier node configuration. To apply the changes, call ApplyAnalogOutSettings() or StartAnalogOut().

Value

Invisibly returns NULL.

See Also

See the vignette *Basic Analog Out Functionality* for a complete Analog Out workflow.

CloseAllDevices *Close all WaveForms devices opened by the current process*

Description

Close all WaveForms device connections opened by the current R process. This operation is process-wide and is not limited to connections opened through **dwf4r**. It may therefore also close connections created by other packages or by direct use of the WaveForms SDK in the same R process. Connections owned by other processes are not affected.

After this function returns, all existing "dwf4r_device" objects and their copies are stale and cannot be used in subsequent **dwf4r** calls. Any WaveForms handles created by other code in the current R process are also invalidated.

Usage

```
CloseAllDevices()
```

Value

Invisibly returns NULL.

Examples

```
## Not run:
devices <- ListDevices()
if (nrow(devices) >= 1L) {
  device_sn <- devices$serial_number[[1L]]
  device <- OpenDevice(device_sn)
  CloseAllDevices()
}

## End(Not run)
```

CloseDevice

Close a device

Description

Close an active WaveForms device connection and mark the corresponding session as closed. After this function returns, the device object and any copies of it are stale and cannot be used in subsequent **dwf4r** calls.

Usage

```
CloseDevice(device)
```

Arguments

device A "dwf4r_device" object returned by `OpenDevice()` that refers to a device connection that is still open.

Value

Invisibly returns NULL.

Examples

```
## Not run:
devices <- ListDevices()
if (nrow(devices) >= 1L) {
  device_sn <- devices$serial_number[[1L]]
  device <- OpenDevice(device_sn)
  CloseDevice(device)
}

## End(Not run)
```

ConfigureAnalogOut	<i>Configure Analog Out generation</i>
--------------------	--

Description

Apply settings, start, or stop Analog Out generation for a selected channel. Applying settings sends updated configuration to the device without changing the current channel state.

ApplyAnalogOutSettings() applies updated Analog Out settings.

StartAnalogOut() starts Analog Out generation.

StopAnalogOut() stops Analog Out generation.

Usage

```
ApplyAnalogOutSettings(device, channel)
```

```
StartAnalogOut(device, channel)
```

```
StopAnalogOut(device, channel)
```

Arguments

device	A "dwf4r_device" object returned by OpenDevice() that refers to a device connection that is still open.
--------	---

channel	An integer scalar representing the zero-based Analog Out channel index.
---------	---

Value

Invisibly returns NULL.

See Also

See the vignette *Basic Analog Out Functionality* for a complete Analog Out workflow.

ConfigureDigitalOut	<i>Start or stop Digital Out</i>
---------------------	----------------------------------

Description

Start or stop the Digital Out instrument.

StartDigitalOut() starts (or restarts) the Digital Out instrument using the configured settings.

StopDigitalOut() stops the Digital Out instrument.

Usage

```
StartDigitalOut(device, .validate_device = TRUE)
```

```
StopDigitalOut(device, .validate_device = TRUE)
```

Arguments

device	A "dwf4r_device" object returned by <code>OpenDevice()</code> that refers to a device connection that is still open.
.validate_device	A logical scalar indicating whether <code>AssertDevice()</code> should be called. This argument is intended for advanced use to avoid repeated validation in controlled sequences of operations. Set to <code>FALSE</code> only when the device has already been validated and is known to remain open. Other argument validation and SDK error handling remain active.

Details

Unlike Analog Out, Digital Out does not provide a separate operation for applying modified settings while preserving the current instrument state. After changing Digital Out settings, use `StartDigitalOut()` to start or restart the instrument with the new configuration.

Value

Invisibly returns `NULL`.

See Also

See the vignette *Basic Digital Out Functionality* for a complete Digital Out workflow.

DigitalOutChannelEnableDisable

Enable or disable a Digital Out channel

Description

Enable or disable a selected Digital Out channel.

`EnableDigitalOutChannel()` enables a Digital Out channel.

`DisableDigitalOutChannel()` disables a Digital Out channel.

Usage

```
EnableDigitalOutChannel(device, channel, .validate_digital_out = TRUE)
```

```
DisableDigitalOutChannel(device, channel, .validate_digital_out = TRUE)
```

Arguments

device	A "dwf4r_device" object returned by <code>OpenDevice()</code> that refers to a device connection that is still open.
channel	An integer scalar representing the zero-based Digital Out channel index.
.validate_digital_out	A logical scalar indicating whether <code>AssertDigitalOut()</code> should be called. This argument is intended for advanced use to avoid repeated validation in controlled sequences of operations. Set to <code>FALSE</code> only when the device and relevant Digital Out context, such as the channel and supported configuration choices, have

already been validated. The caller is responsible for ensuring that the device remains open and the validated context remains valid. Other argument validation and SDK error handling remain active.

Details

This function updates the pending Digital Out configuration. Because automatic configuration is disabled by **dwf4r**, call `StartDigitalOut()` to start or restart generation with the updated settings. To stop generation, call `StopDigitalOut()` explicitly.

Value

Invisibly returns NULL.

See Also

See the vignette *Basic Digital Out Functionality* for a complete Digital Out workflow.

GetAnalogOutIdle

Get the Analog Out idle output mode

Description

Retrieve the idle output mode currently configured for a selected Analog Out channel.

The idle mode controls the output while the generator is not actively running, including the Ready, Stopped, Done, and Wait states.

Usage

```
GetAnalogOutIdle(device, channel)
```

Arguments

device	A "dwf4r_device" object returned by <code>OpenDevice()</code> that refers to a device connection that is still open.
channel	An integer scalar representing the zero-based Analog Out channel index.

Value

A string containing "disable", "offset", "initial", or "hold".

See Also

See the vignette *Basic Analog Out Functionality* for a complete Analog Out workflow.

GetAnalogOutInfo

Get Analog Out characteristics

Description

Retrieve Analog Out characteristics of a device.

GetAnalogOutChannelCount() returns the number of Analog Out channels.

GetAnalogOutChannelNodes() returns the nodes supported by an Analog Out channel.

GetAnalogOutRunRange() returns the supported run-time range, in seconds, for an Analog Out channel.

GetAnalogOutWaitRange() returns the supported wait-time range, in seconds, for an Analog Out channel.

GetAnalogOutRepeatRange() returns the supported repeat-count range for an Analog Out channel.

GetAnalogOutIdleModes() returns the idle output modes supported by an Analog Out channel.

GetAnalogOutNodeFunctionTypes() returns the waveform functions supported by an Analog Out node.

GetAnalogOutNodeFrequencyRange() returns the supported frequency range for an Analog Out node.

GetAnalogOutNodeAmplitudeRange() returns the supported amplitude range for an Analog Out node.

GetAnalogOutNodeOffsetRange() returns the supported offset range for an Analog Out node.

GetAnalogOutNodeSymmetryRange() returns the supported symmetry range for an Analog Out node.

GetAnalogOutNodePhaseRange() returns the supported phase range, in degrees, for an Analog Out node.

GetAnalogOutNodeSampleCountRange() returns the supported range of sample counts for custom waveform data.

Usage

```
GetAnalogOutChannelCount(device)
```

```
GetAnalogOutChannelNodes(device, channel)
```

```
GetAnalogOutRunRange(device, channel)
```

```
GetAnalogOutWaitRange(device, channel)
```

```
GetAnalogOutRepeatRange(device, channel)
```

```
GetAnalogOutIdleModes(device, channel)
```

```
GetAnalogOutNodeFunctionTypes(device, channel, node)
```

```
GetAnalogOutNodeFrequencyRange(device, channel, node)
```

```
GetAnalogOutNodeAmplitudeRange(device, channel, node)
```

```
GetAnalogOutNodeOffsetRange(device, channel, node)
```

```
GetAnalogOutNodeSymmetryRange(device, channel, node)
```

```
GetAnalogOutNodePhaseRange(device, channel, node)
```

```
GetAnalogOutNodeSampleCountRange(device, channel, node)
```

Arguments

device	A "dwf4r_device" object returned by OpenDevice() that refers to a device connection that is still open.
channel	An integer scalar representing the zero-based Analog Out channel index.
node	A string specifying the Analog Out node. One of "carrier", "fm", or "am".

Value

An integer, numeric, or character vector describing the requested Analog Out capability. Range functions return two elements containing the minimum and maximum supported values.

See Also

See the vignette *Basic Analog Out Functionality* for a complete Analog Out workflow.

GetAnalogOutMaster	<i>Get the Analog Out master channel</i>
--------------------	--

Description

Retrieve the state-machine master currently configured for a selected Analog Out channel.

Usage

```
GetAnalogOutMaster(device, channel)
```

Arguments

device	A "dwf4r_device" object returned by OpenDevice() that refers to a device connection that is still open.
channel	An integer scalar representing the zero-based Analog Out channel index.

Value

An integer scalar containing the zero-based index of the configured master channel. If the returned index is equal to channel, the channel reports itself as its state-machine master. On Analog Discovery 2, this configuration was observed experimentally to correspond to independent channel operation, although this behavior is not explicitly described in the WaveForms SDK reference manual.

See Also

See the vignette *Basic Analog Out Functionality* for a complete Analog Out workflow.

GetAnalogOutRemaining *Get remaining Analog Out timing values*

Description

Retrieve the remaining run time or repeat count for a selected Analog Out channel. The remaining wait time cannot be retrieved because the WaveForms SDK does not provide a corresponding Analog Out wait-status function.

GetAnalogOutRemainingRun() returns the remaining run time of an Analog Out channel.

GetAnalogOutRemainingRepeat() returns the remaining repeat count of an Analog Out channel.

Usage

```
GetAnalogOutRemainingRun(device, channel, update = TRUE)
```

```
GetAnalogOutRemainingRepeat(device, channel, update = TRUE)
```

Arguments

device	A "dwf4r_device" object returned by OpenDevice() that refers to a device connection that is still open.
channel	An integer scalar representing the zero-based Analog Out channel index.
update	A logical scalar indicating whether the Analog Out status should be updated before retrieving the remaining value. The default is TRUE. If FALSE, the value from the most recent Analog Out status update (performed with GetAnalogOutStatus()) is returned.

Details

The WaveForms SDK functions used to retrieve the remaining run time and repeat count do not themselves read the device. They return information obtained by the most recent GetAnalogOutStatus() (i.e. FDwfAnalogOutStatus()) call.

With the default update = TRUE, these functions update the Analog Out status before retrieving the remaining value. With update = FALSE, no status update is performed and the returned value may therefore be stale.

Setting update = FALSE can also be used to retrieve several values from the same status update. For example, first call GetAnalogOutStatus(), then call both remaining-value functions with update = FALSE.

Value

GetAnalogOutRemainingRun() returns a numeric scalar containing the remaining run time in seconds. GetAnalogOutRemainingRepeat() returns an integer scalar containing the remaining repeat count.

See Also

See the vignette *Basic Analog Out Functionality* for a complete Analog Out workflow.

`GetAnalogOutRepeatTrigger`*Get Analog Out repeat triggering*

Description

Retrieve whether a new trigger is included in each repeat cycle of a selected Analog Out channel.

Usage

```
GetAnalogOutRepeatTrigger(device, channel)
```

Arguments

device	A "dwf4r_device" object returned by <code>OpenDevice()</code> that refers to a device connection that is still open.
channel	An integer scalar representing the zero-based Analog Out channel index.

Value

A logical scalar. TRUE indicates that a new trigger is included in each wait-run repeat cycle; FALSE indicates that repeated cycles proceed without waiting for another trigger.

`GetAnalogOutStatus`*Get the state of an Analog Out channel*

Description

Retrieve the current state of a selected Analog Out channel.

Usage

```
GetAnalogOutStatus(device, channel)
```

Arguments

device	A "dwf4r_device" object returned by <code>OpenDevice()</code> that refers to a device connection that is still open.
channel	An integer scalar representing the zero-based Analog Out channel index.

Value

A string containing "ready", "armed", "wait", "running", "done", "config", "prefill", or "not_done". The value "unknown" is returned for an unrecognized SDK state.

See Also

See the vignette *Basic Analog Out Functionality* for a complete Analog Out workflow.

GetAnalogOutTiming	<i>Get Analog Out timing</i>
--------------------	------------------------------

Description

Retrieve the configured run time, post-trigger wait time, or repeat count for a selected Analog Out channel.

GetAnalogOutRun() returns the configured run length of an Analog Out channel.

GetAnalogOutWait() returns the configured delay between receiving a trigger and beginning signal generation.

GetAnalogOutRepeat() returns the configured repeat count of an Analog Out channel.

GetAnalogOutTiming() returns all configured timing parameters of an Analog Out channel.

Usage

```
GetAnalogOutRun(device, channel)
```

```
GetAnalogOutWait(device, channel)
```

```
GetAnalogOutRepeat(device, channel)
```

```
GetAnalogOutTiming(device, channel)
```

Arguments

device	A "dwf4r_device" object returned by OpenDevice() that refers to a device connection that is still open.
--------	---

channel	An integer scalar representing the zero-based Analog Out channel index.
---------	---

Details

These functions retrieve the configured timing values reported by the corresponding WaveForms SDK *Get functions. A run time of 0 represents continuous generation, a wait time of 0 represents no post-trigger delay, and a repeat count of 0 represents infinite repetition.

Value

GetAnalogOutRun() and GetAnalogOutWait() return a numeric scalar in seconds. GetAnalogOutRepeat() returns an integer scalar. GetAnalogOutTiming() returns a named list with the following elements:

run_time Numeric configured run time in seconds.

wait_time Numeric configured post-trigger wait time in seconds.

repeat_count Integer configured repeat count.

See Also

See the vignette *Basic Analog Out Functionality* for a complete Analog Out workflow.

`GetAnalogOutTriggerSlope`*Get the Analog Out trigger slope*

Description

Retrieve the trigger slope currently configured for a selected Analog Out channel.

Usage

```
GetAnalogOutTriggerSlope(device, channel)
```

Arguments

device	A "dwf4r_device" object returned by <code>OpenDevice()</code> that refers to a device connection that is still open.
channel	An integer scalar representing the zero-based Analog Out channel index.

Value

A string identifying the configured trigger slope.

`GetAnalogOutTriggerSource`*Get the Analog Out trigger source*

Description

Retrieve the trigger source currently configured for a selected Analog Out channel.

Usage

```
GetAnalogOutTriggerSource(device, channel)
```

Arguments

device	A "dwf4r_device" object returned by <code>OpenDevice()</code> that refers to a device connection that is still open.
channel	An integer scalar representing the zero-based Analog Out channel index.

Value

A string identifying the configured trigger source.

GetCarrierFunction	<i>Get the carrier waveform function</i>
--------------------	--

Description

Retrieve the waveform function currently configured for the carrier node of an Analog Out channel.

Usage

```
GetCarrierFunction(device, channel)
```

Arguments

device	A "dwf4r_device" object returned by OpenDevice() that refers to a device connection that is still open.
channel	An integer scalar representing the zero-based Analog Out channel index.

Value

A string identifying the configured waveform function. Possible values can be queried with GetAnalogOutNodeFunction

See Also

See the vignette *Basic Analog Out Functionality* for a complete Analog Out workflow.

GetCarrierSettings	<i>Get carrier waveform settings</i>
--------------------	--------------------------------------

Description

Retrieve the current configuration of the carrier node for an Analog Out channel.

Usage

```
GetCarrierSettings(device, channel)
```

Arguments

device	A "dwf4r_device" object returned by OpenDevice() that refers to a device connection that is still open.
channel	An integer scalar representing the zero-based Analog Out channel index.

Value

A named list with the following elements:

`enabled` Logical value indicating whether the carrier node is enabled.

`func` String identifying the configured waveform function.

`frequency` Numeric carrier frequency in Hz.

`amplitude` Numeric carrier peak amplitude in volts relative to the configured offset.

`offset` Numeric carrier offset in volts.

`symmetry` Numeric carrier symmetry in percent.

`phase` Numeric carrier phase in degrees.

See Also

See the vignette *Basic Analog Out Functionality* for a complete Analog Out workflow.

GetCarrierValues	<i>Get numeric carrier waveform parameters</i>
------------------	--

Description

Retrieve numeric parameters currently configured for the carrier node of an Analog Out channel.

`GetCarrierFrequency()` returns the configured carrier frequency in Hz.

`GetCarrierAmplitude()` returns the configured carrier peak amplitude in volts relative to the configured offset.

`GetCarrierOffset()` returns the configured carrier offset in volts.

`GetCarrierSymmetry()` returns the configured carrier symmetry in percent.

`GetCarrierPhase()` returns the configured carrier phase in degrees.

Usage

```
GetCarrierFrequency(device, channel)
```

```
GetCarrierAmplitude(device, channel)
```

```
GetCarrierOffset(device, channel)
```

```
GetCarrierSymmetry(device, channel)
```

```
GetCarrierPhase(device, channel)
```

Arguments

`device` A "dwf4r_device" object returned by `OpenDevice()` that refers to a device connection that is still open.

`channel` An integer scalar representing the zero-based Analog Out channel index.

Details

The functions return values reported by the corresponding WaveForms SDK *Get functions. A returned value may differ slightly from the value supplied to a setter if the SDK adjusted it to a value supported by the device.

Value

A numeric scalar containing the requested carrier parameter.

See Also

See the vignette *Basic Analog Out Functionality* for a complete Analog Out workflow.

GetDeviceTriggerInfo *Get device trigger capabilities*

Description

Retrieve trigger capabilities of a WaveForms device.

GetDeviceTriggerSources() returns the trigger sources supported by the device and available through the current **dwf4r** interface.

Possible values include "none", "pc", "detector_analog_in", "detector_digital_in", "analog_in", "digital_in", "digital_out", "analog_out_1", "analog_out_2", "external_1", and "external_2".

GetDeviceTriggerSlopes() returns the trigger slopes supported by the device. Possible values are "rising", "falling", and "either".

Usage

```
GetDeviceTriggerSources(device, .validate_device = TRUE)
```

```
GetDeviceTriggerSlopes(device, .validate_device = TRUE)
```

Arguments

device	A "dwf4r_device" object returned by OpenDevice() that refers to a device connection that is still open.
.validate_device	A logical scalar indicating whether AssertDevice() should be called. This argument is intended for advanced use to avoid repeated validation in controlled sequences of operations. Set to FALSE only when the device has already been validated and is known to remain open. Other argument validation and SDK error handling remain active.

Value

A character vector containing supported trigger source or slope names, depending on the function.

GetDigitalOutChannelInfo

Get Digital Out channel characteristics

Description

Retrieve Digital Out characteristics of a selected channel.

GetDigitalOutOutputModes() returns the output modes supported by a Digital Out channel.

GetDigitalOutFunctionTypes() returns the signal-generation types supported by a Digital Out channel.

GetDigitalOutIdleModes() returns the idle output modes supported by a Digital Out channel.

GetDigitalOutDividerRange() returns the supported divider range for a Digital Out channel.

GetDigitalOutCounterRange() returns the supported counter range for a Digital Out channel.

GetDigitalOutMaxDataBits() returns the maximum number of custom-data bits supported by a Digital Out channel.

Usage

```
GetDigitalOutOutputModes(device, channel, .validate_digital_out = TRUE)
```

```
GetDigitalOutFunctionTypes(device, channel, .validate_digital_out = TRUE)
```

```
GetDigitalOutIdleModes(device, channel, .validate_digital_out = TRUE)
```

```
GetDigitalOutDividerRange(device, channel, .validate_digital_out = TRUE)
```

```
GetDigitalOutCounterRange(device, channel, .validate_digital_out = TRUE)
```

```
GetDigitalOutMaxDataBits(device, channel, .validate_digital_out = TRUE)
```

Arguments

device A "dwf4r_device" object returned by OpenDevice() that refers to a device connection that is still open.

channel An integer scalar representing the zero-based Digital Out channel index.

.validate_digital_out

A logical scalar indicating whether AssertDigitalOut() should be called. This argument is intended for advanced use to avoid repeated validation in controlled sequences of operations. Set to FALSE only when the device and relevant Digital Out context, such as the channel and supported configuration choices, have already been validated. The caller is responsible for ensuring that the device remains open and the validated context remains valid. Other argument validation and SDK error handling remain active.

Value

An integer, numeric, or character vector describing the requested Digital Out capability. Range functions return two elements containing the minimum and maximum supported values. Capability values originating from unsigned SDK integers are returned as integers when all values can be represented by R integers; otherwise they are returned as numeric whole-number values.

See Also

See the vignette *Basic Digital Out Functionality* for a complete Digital Out workflow.

GetDigitalOutChannelSettings

Get Digital Out channel settings

Description

Retrieve the output mode, signal-generation function, or idle output mode currently configured for a selected Digital Out channel.

GetDigitalOutOutput() returns the configured output mode of a Digital Out channel.

GetDigitalOutFunction() returns the configured signal-generation function of a Digital Out channel.

GetDigitalOutIdle() returns the configured idle output mode of a Digital Out channel.

Usage

```
GetDigitalOutOutput(device, channel, .validate_digital_out = TRUE)
```

```
GetDigitalOutFunction(device, channel, .validate_digital_out = TRUE)
```

```
GetDigitalOutIdle(device, channel, .validate_digital_out = TRUE)
```

Arguments

device A "dwf4r_device" object returned by OpenDevice() that refers to a device connection that is still open.

channel An integer scalar representing the zero-based Digital Out channel index.

.validate_digital_out

A logical scalar indicating whether AssertDigitalOut() should be called. This argument is intended for advanced use to avoid repeated validation in controlled sequences of operations. Set to FALSE only when the device and relevant Digital Out context, such as the channel and supported configuration choices, have already been validated. The caller is responsible for ensuring that the device remains open and the validated context remains valid. Other argument validation and SDK error handling remain active.

Details

The WaveForms SDK and the public **dwf4r** API use different terms for selecting the type of generated signal. The SDK uses *type*, whereas the R API uses *function*. The latter is used primarily to maintain consistent naming between Analog Out and Digital Out functionality.

Value

A string identifying the configured Digital Out channel setting. Possible values for each setting can be queried with the corresponding capability function.

See Also

See the vignette *Basic Digital Out Functionality* for a complete Digital Out workflow.

GetDigitalOutCounter *Get Digital Out low and high counters*

Description

Retrieve the low and high counter values configured for a selected Digital Out channel.

Usage

```
GetDigitalOutCounter(device, channel, .validate_digital_out = TRUE)
```

Arguments

device	A "dwf4r_device" object returned by <code>OpenDevice()</code> that refers to a device connection that is still open.
channel	An integer scalar representing the zero-based Digital Out channel index.
.validate_digital_out	A logical scalar indicating whether <code>AssertDigitalOut()</code> should be called. This argument is intended for advanced use to avoid repeated validation in controlled sequences of operations. Set to <code>FALSE</code> only when the device and relevant Digital Out context, such as the channel and supported configuration choices, have already been validated. The caller is responsible for ensuring that the device remains open and the validated context remains valid. Other argument validation and SDK error handling remain active.

Details

The low and high counters determine how many counts the output remains low and high, respectively. The internal clock frequency and divider value determine the duration of one count.

The function returns the values reported by the corresponding WaveForms SDK `*Get` function.

Value

A named vector containing `low_count` and `high_count`. Values are returned as integers when both can be represented by R integers; otherwise they are returned as numeric whole-number values.

See Also

See the vignette *Basic Digital Out Functionality* for a complete Digital Out workflow.

`GetDigitalOutCustomSampleRate`*Get the Digital Out custom-pattern sample rate*

Description

Retrieve the custom-pattern sample rate corresponding to the ordinary divider configured for a selected Digital Out channel.

Usage

```
GetDigitalOutCustomSampleRate(device, channel, .validate_digital_out = TRUE)
```

Arguments

<code>device</code>	A "dwf4r_device" object returned by <code>OpenDevice()</code> that refers to a device connection that is still open.
<code>channel</code>	An integer scalar representing the zero-based Digital Out channel index.
<code>.validate_digital_out</code>	A logical scalar indicating whether <code>AssertDigitalOut()</code> should be called. This argument is intended for advanced use to avoid repeated validation in controlled sequences of operations. Set to <code>FALSE</code> only when the device and relevant Digital Out context, such as the channel and supported configuration choices, have already been validated. The caller is responsible for ensuring that the device remains open and the validated context remains valid. Other argument validation and SDK error handling remain active.

Details

The sample rate is calculated as the Digital Out internal clock frequency divided by the configured ordinary divider.

Value

A numeric scalar containing the custom-pattern sample rate in Hz. The sample rate is the rate at which individual pattern elements are generated, not the repetition frequency of the complete pattern. For example, at a sample rate of 1 kHz, each pattern element lasts 1 ms.

See Also

See the vignette *Basic Digital Out Functionality* for a complete Digital Out workflow.

GetDigitalOutDividerValues

Get Digital Out divider values

Description

Retrieve the ordinary or initial divider configured for a selected Digital Out channel. The internal clock frequency and divider value determine the duration of one count. The initial divider is loaded when the instrument enters the Running state; after it expires, the ordinary divider is loaded on subsequent expirations.

GetDigitalOutDivider() returns the configured ordinary divider.

GetDigitalOutInitialDivider() returns the configured initial divider.

Usage

```
GetDigitalOutDivider(device, channel, .validate_digital_out = TRUE)
```

```
GetDigitalOutInitialDivider(device, channel, .validate_digital_out = TRUE)
```

Arguments

device	A "dwf4r_device" object returned by OpenDevice() that refers to a device connection that is still open.
channel	An integer scalar representing the zero-based Digital Out channel index.
.validate_digital_out	A logical scalar indicating whether AssertDigitalOut() should be called. This argument is intended for advanced use to avoid repeated validation in controlled sequences of operations. Set to FALSE only when the device and relevant Digital Out context, such as the channel and supported configuration choices, have already been validated. The caller is responsible for ensuring that the device remains open and the validated context remains valid. Other argument validation and SDK error handling remain active.

Details

The functions return the values reported by the corresponding WaveForms SDK *Get functions.

Value

An integer scalar containing the configured divider value, or a numeric whole-number scalar if the value exceeds R's integer range.

See Also

See the vignette *Basic Digital Out Functionality* for a complete Digital Out workflow.

GetDigitalOutInfo	<i>Get Digital Out instrument characteristics</i>
-------------------	---

Description

Retrieve Digital Out characteristics of a device.

GetDigitalOutChannelCount() returns the number of Digital Out channels.

GetDigitalOutInternalClockFrequency() returns the internal Digital Out clock frequency in Hz.

GetDigitalOutRunRange() returns the supported finite run-time range, in seconds. A run time of zero is a special SDK value specifying continuous generation.

GetDigitalOutWaitRange() returns the supported nonzero wait-time range, in seconds. A wait time of zero specifies no wait and is the SDK default.

GetDigitalOutRepeatRange() returns the supported repeat-count range. A repeat count of zero is a special SDK value specifying infinite repeats.

Usage

```
GetDigitalOutChannelCount(device, .validate_device = TRUE)
```

```
GetDigitalOutInternalClockFrequency(device, .validate_device = TRUE)
```

```
GetDigitalOutRunRange(device, .validate_device = TRUE)
```

```
GetDigitalOutWaitRange(device, .validate_device = TRUE)
```

```
GetDigitalOutRepeatRange(device, .validate_device = TRUE)
```

Arguments

device A "dwf4r_device" object returned by OpenDevice() that refers to a device connection that is still open.

.validate_device A logical scalar indicating whether AssertDevice() should be called. This argument is intended for advanced use to avoid repeated validation in controlled sequences of operations. Set to FALSE only when the device has already been validated and is known to remain open. Other argument validation and SDK error handling remain active.

Value

An integer or numeric vector describing the requested Digital Out capability. Range functions return two elements containing the minimum and maximum supported values. Capability values originating from unsigned SDK integers are returned as integers when all values can be represented by R integers; otherwise they are returned as numeric whole-number values.

See Also

See the vignette *Basic Digital Out Functionality* for a complete Digital Out workflow.

GetDigitalOutInitialCounter

Get the initial Digital Out counter

Description

Retrieve the initial output state and counter value configured for a selected Digital Out channel.

Usage

```
GetDigitalOutInitialCounter(device, channel, .validate_digital_out = TRUE)
```

Arguments

device	A "dwf4r_device" object returned by <code>OpenDevice()</code> that refers to a device connection that is still open.
channel	An integer scalar representing the zero-based Digital Out channel index.
.validate_digital_out	A logical scalar indicating whether <code>AssertDigitalOut()</code> should be called. This argument is intended for advanced use to avoid repeated validation in controlled sequences of operations. Set to <code>FALSE</code> only when the device and relevant Digital Out context, such as the channel and supported configuration choices, have already been validated. The caller is responsible for ensuring that the device remains open and the validated context remains valid. Other argument validation and SDK error handling remain active.

Details

The initial state and counter value are loaded when the Digital Out instrument enters the Running state. The initial counter determines the number of counts before the ordinary low and high counter sequence begins.

The function returns the values reported by the corresponding WaveForms SDK `*Get` function.

Value

A named list with the following elements:

`initial_state` A string, either "low" or "high", identifying the configured initial output state.

`initial_count` The configured initial counter value, returned as an integer or a numeric whole-number scalar if it exceeds R's integer range.

See Also

See the vignette *Basic Digital Out Functionality* for a complete Digital Out workflow.

GetDigitalOutRemaining

Get remaining Digital Out timing values

Description

Retrieve the remaining run value or repeat count for the Digital Out instrument. The remaining wait time cannot be retrieved because the WaveForms SDK does not provide a corresponding Digital Out wait-status function.

GetDigitalOutRemainingRun() returns the remaining run value reported by the Digital Out instrument.

GetDigitalOutRemainingRepeat() returns the remaining repeat count of the Digital Out instrument.

Usage

```
GetDigitalOutRemainingRun(device, update = TRUE, .validate_device = TRUE)
```

```
GetDigitalOutRemainingRepeat(device, update = TRUE, .validate_device = TRUE)
```

Arguments

device	A "dwf4r_device" object returned by OpenDevice() that refers to a device connection that is still open.
update	A logical scalar indicating whether the Digital Out status should be updated before retrieving the remaining value. The default is TRUE. If FALSE, the value from the most recent Digital Out status update is returned.
.validate_device	A logical scalar indicating whether AssertDevice() should be called. This argument is intended for advanced use to avoid repeated validation in controlled sequences of operations. Set to FALSE only when the device has already been validated and is known to remain open. Other argument validation and SDK error handling remain active.

Details

The WaveForms SDK functions used to retrieve the remaining run time and repeat count do not themselves read the device. They return information obtained by the most recent FDwfDigitalOutStatus() call, performed by GetDigitalOutStatus() or by either remaining-value function with update = TRUE.

With the default update = TRUE, these functions update the Digital Out status before retrieving the remaining value. With update = FALSE, no status update is performed and the returned value may therefore be stale.

To retrieve both remaining values from the same status update, first call GetDigitalOutStatus(), then call both remaining-value functions with update = FALSE.

These functions preserve the values reported by the SDK. They do not convert values to Inf or infer completion from a remaining value of zero. Use GetDigitalOutStatus() to determine the instrument state and the configured timing getters to identify continuous generation or infinite repetition.

The WaveForms SDK does not document the units returned by `FDwfDigitalOutRunStatus()`. On Analog Discovery 2, the returned value was found to correspond to Digital Out internal-clock ticks. This behavior has not been verified for other devices.

Value

`GetDigitalOutRemainingRun()` returns a numeric scalar containing the remaining run value reported by the SDK. For Analog Discovery 2, this value is reported in Digital Out internal-clock ticks; the units have not been verified for other devices.

`GetDigitalOutRemainingRepeat()` returns the remaining repeat count as an integer scalar when it is no greater than `.Machine$integer.max`; otherwise it returns a numeric scalar to preserve the full unsigned SDK range.

See Also

See the vignette *Basic Digital Out Functionality* for a complete Digital Out workflow.

GetDigitalOutRepeatTrigger

Get Digital Out repeat triggering

Description

Retrieve whether a trigger is included in each wait-run repeat cycle of the Digital Out instrument. This setting is shared by all Digital Out channels.

Usage

```
GetDigitalOutRepeatTrigger(device, .validate_device = TRUE)
```

Arguments

<code>device</code>	A "dwf4r_device" object returned by <code>OpenDevice()</code> that refers to a device connection that is still open.
<code>.validate_device</code>	A logical scalar indicating whether <code>AssertDevice()</code> should be called. This argument is intended for advanced use to avoid repeated validation in controlled sequences of operations. Set to <code>FALSE</code> only when the device has already been validated and is known to remain open. Other argument validation and SDK error handling remain active.

Value

A logical scalar. `TRUE` indicates that subsequent repeat cycles return to the Armed state before proceeding through Wait and Running; `FALSE` indicates that they proceed without waiting for another trigger. This is the configured setting, not the current instrument state.

GetDigitalOutStatus *Get the state of the Digital Out instrument*

Description

Retrieve the current state of the Digital Out instrument.

Usage

```
GetDigitalOutStatus(device, .validate_device = TRUE)
```

Arguments

device	A "dwf4r_device" object returned by OpenDevice() that refers to a device connection that is still open.
.validate_device	A logical scalar indicating whether AssertDevice() should be called. This argument is intended for advanced use to avoid repeated validation in controlled sequences of operations. Set to FALSE only when the device has already been validated and is known to remain open. Other argument validation and SDK error handling remain active.

Value

A string identifying the current Digital Out state. The WaveForms SDK documents "ready", "armed", "wait", "running", and "done" as Digital Out states. The additional SDK states "config", "prefill", and "not_done" are also recognized if returned. The value "unknown" is returned for any unrecognized SDK state.

See Also

See the vignette *Basic Digital Out Functionality* for a complete Digital Out workflow.

GetDigitalOutTiming *Get Digital Out timing*

Description

Retrieve the configured run time, post-trigger wait time, or repeat count for the Digital Out instrument. These settings are shared by all Digital Out channels.

GetDigitalOutRun() returns the configured run length of the Digital Out instrument.

GetDigitalOutWait() returns the configured delay between receiving a trigger and beginning signal generation.

GetDigitalOutRepeat() returns the configured repeat count of the Digital Out instrument.

GetDigitalOutTiming() returns all configured timing parameters of the Digital Out instrument.

Usage

```

GetDigitalOutRun(device, .validate_device = TRUE)

GetDigitalOutWait(device, .validate_device = TRUE)

GetDigitalOutRepeat(device, .validate_device = TRUE)

GetDigitalOutTiming(device, .validate_device = TRUE)

```

Arguments

device	A "dwf4r_device" object returned by <code>OpenDevice()</code> that refers to a device connection that is still open.
.validate_device	A logical scalar indicating whether <code>AssertDevice()</code> should be called. This argument is intended for advanced use to avoid repeated validation in controlled sequences of operations. Set to <code>FALSE</code> only when the device has already been validated and is known to remain open. Other argument validation and SDK error handling remain active.

Details

These functions retrieve the configured timing values reported by the corresponding WaveForms SDK *Get functions. A run time of 0 represents continuous generation, a wait time of 0 represents no post-trigger delay, and a repeat count of 0 represents infinite repetition.

Configured values are distinct from remaining run time and repeat count. Use `GetDigitalOutRemainingRun()` and `GetDigitalOutRemainingRepeat()` to retrieve the remaining values.

Value

`GetDigitalOutRun()` and `GetDigitalOutWait()` return numeric scalars in seconds. `GetDigitalOutRepeat()` returns an integer scalar when the count is no greater than `.Machine$integer.max`; otherwise it returns a numeric scalar to preserve the full unsigned SDK range.

`GetDigitalOutTiming()` returns a named list with the following elements:

`run_time` Numeric configured run time in seconds.

`wait_time` Numeric configured post-trigger wait time in seconds.

`repeat_count` Integer or numeric configured repeat count, using the same conversion as `GetDigitalOutRepeat()`.

See Also

See the vignette *Basic Digital Out Functionality* for a complete Digital Out workflow.

GetDigitalOutTriggerSlope

Get the Digital Out trigger slope

Description

Retrieve the trigger slope currently configured for the Digital Out instrument. This setting is shared by all Digital Out channels.

Usage

```
GetDigitalOutTriggerSlope(device, .validate_device = TRUE)
```

Arguments

device	A "dwf4r_device" object returned by OpenDevice() that refers to a device connection that is still open.
.validate_device	A logical scalar indicating whether AssertDevice() should be called. This argument is intended for advanced use to avoid repeated validation in controlled sequences of operations. Set to FALSE only when the device has already been validated and is known to remain open. Other argument validation and SDK error handling remain active.

Value

A string identifying the configured trigger slope: "rising", "falling", or "either". An error is raised if the SDK returns a slope code not recognized by **dwf4r**.

GetDigitalOutTriggerSource

Get the Digital Out trigger source

Description

Retrieve the trigger source currently configured for the Digital Out instrument. This setting is shared by all Digital Out channels.

Usage

```
GetDigitalOutTriggerSource(device, .validate_device = TRUE)
```

Arguments

device	A "dwf4r_device" object returned by OpenDevice() that refers to a device connection that is still open.
.validate_device	A logical scalar indicating whether AssertDevice() should be called. This argument is intended for advanced use to avoid repeated validation in controlled sequences of operations. Set to FALSE only when the device has already been validated and is known to remain open. Other argument validation and SDK error handling remain active.

Value

A string identifying the configured trigger source. An error is raised if the SDK returns a source code not recognized by **dwf4r**.

IsCarrierEnabled	<i>Check whether carrier output is enabled</i>
------------------	--

Description

Retrieve the configured enable state of the carrier node for an Analog Out channel.

Usage

```
IsCarrierEnabled(device, channel)
```

Arguments

device	A "dwf4r_device" object returned by OpenDevice() that refers to a device connection that is still open.
channel	An integer scalar representing the zero-based Analog Out channel index.

Value

A logical scalar, TRUE if the carrier node is enabled and FALSE if it is disabled.

See Also

See the vignette *Basic Analog Out Functionality* for a complete Analog Out workflow.

IsDigitalOutChannelEnabled	<i>Check whether a Digital Out channel is enabled</i>
----------------------------	---

Description

Retrieve the configured enable state of a selected Digital Out channel.

Usage

```
IsDigitalOutChannelEnabled(device, channel, .validate_digital_out = TRUE)
```

Arguments

- device A "dwf4r_device" object returned by OpenDevice() that refers to a device connection that is still open.
- channel An integer scalar representing the zero-based Digital Out channel index.
- .validate_digital_out A logical scalar indicating whether AssertDigitalOut() should be called. This argument is intended for advanced use to avoid repeated validation in controlled sequences of operations. Set to FALSE only when the device and relevant Digital Out context, such as the channel and supported configuration choices, have already been validated. The caller is responsible for ensuring that the device remains open and the validated context remains valid. Other argument validation and SDK error handling remain active.

Value

A logical scalar, TRUE if the channel is enabled and FALSE if it is disabled.

See Also

See the vignette *Basic Digital Out Functionality* for a complete Digital Out workflow.

ListDevices	List detected devices
-------------	-----------------------

Description

Enumerate detected devices using the filters specified by filters and retrieve some additional information for each device.

Usage

ListDevices(filters = "all")

Arguments

- filters A character vector specifying one or more enumeration filters. The default, "all", corresponds to the SDK's default enumeration filter. Other supported values are "usb", "network", "axi", "remote", "audio", and "demo".

Value

- A data frame with one row per enumerated device and the following columns:
- device_index Zero-based index of the device in the current SDK enumeration result.
 - device_id Integer device-type identifier defined by the SDK.
 - device_type Character label mapped from device_id; unknown SDK identifiers are reported as "Unknown".
 - device_revision Integer device-revision identifier.
 - device_name Character string containing the device name.
 - user_name Character string containing the user-assigned device name.

`serial_number` Character string containing the unique device serial number.

`is_opened` Logical value indicating whether the device is already opened by the current or another process.

`config_count` Integer number of available configurations for the device.

Examples

```
## Not run:
devices <- ListDevices()
print(devices)

usb_devices <- ListDevices(filters = "usb")
print(usb_devices)

## End(Not run)
```

OpenDevice	<i>Open a device</i>
------------	----------------------

Description

Open a WaveForms device, register the connection in the current R session, and return an object that can be passed to other **dwf4r** functions.

Usage

```
OpenDevice(serial_number = NULL)
```

Arguments

`serial_number` A string containing the serial number of the device to open. If `NULL`, the device is selected automatically when exactly one compatible device is detected.

Details

The device is opened using its default configuration. Automatic configuration in the WaveForms SDK is disabled to reduce the communication overhead of individual setting calls. Instrument settings are therefore applied explicitly by the corresponding **dwf4r** configuration functions. This behavior is managed internally and is not currently user-configurable.

Value

An opaque object of class `"dwf4r_device"` representing the opened connection. Pass this object unchanged to other **dwf4r** functions.

Examples

```
## Not run:
devices <- ListDevices()

if (nrow(devices) >= 1L) {
  if (nrow(devices) == 1L) {
    device <- OpenDevice()
  } else {
    device_sn <- devices$serial_number[[1L]]
    device <- OpenDevice(device_sn)
  }

  print(device)

  CloseDevice(device)
}

## End(Not run)
```

ResetAnalogOut

*Reset Analog Out channel***Description**

Reset the Analog Out settings of a selected channel to their SDK defaults. Because automatic configuration is disabled by **dwf4r**, call `ApplyAnalogOutSettings()` or `StartAnalogOut()` to apply the reset configuration to the device.

Usage

```
ResetAnalogOut(device, channel)
```

Arguments

device	A "dwf4r_device" object returned by <code>OpenDevice()</code> that refers to a device connection that is still open.
channel	An integer scalar representing the zero-based Analog Out channel index.

Value

Invisibly returns NULL.

See Also

See the vignette *Basic Analog Out Functionality* for a complete Analog Out workflow.

ResetDigitalOut	<i>Reset Digital Out instrument</i>
-----------------	-------------------------------------

Description

Reset all Digital Out instrument parameters to their SDK defaults. Because automatic configuration is disabled by **dwf4r**, call StartDigitalOut() or StopDigitalOut() after resetting, depending on the desired instrument state.

Usage

```
ResetDigitalOut(device, .validate_device = TRUE)
```

Arguments

device	A "dwf4r_device" object returned by OpenDevice() that refers to a device connection that is still open.
.validate_device	A logical scalar indicating whether AssertDevice() should be called. This argument is intended for advanced use to avoid repeated validation in controlled sequences of operations. Set to FALSE only when the device has already been validated and is known to remain open. Other argument validation and SDK error handling remain active.

Value

Invisibly returns NULL.

See Also

See the vignette *Basic Digital Out Functionality* for a complete Digital Out workflow.

SetAnalogOutIdle	<i>Set Analog Out idle output</i>
------------------	-----------------------------------

Description

Set the idle output mode for a selected Analog Out channel.

The idle output mode controls the Analog Out output while the generator is not actively running, including Ready, Stopped, Done, and Wait states.

Usage

```
SetAnalogOutIdle(device, channel, idle)
```

Arguments

device	A "dwf4r_device" object returned by OpenDevice() that refers to a device connection that is still open.
channel	An integer scalar representing the zero-based Analog Out channel index.
idle	A string specifying the idle output mode. Supported values are "disable", "offset", "initial", and "hold".

Details

The "offset" mode uses the configured offset level. Use SetCarrierOffset() to set that voltage level. If device auto-configuration is disabled, call ApplyAnalogOutSettings() or StartAnalogOut() after changing Analog Out settings.

Value

Invisibly returns NULL.

See Also

See the vignette *Basic Analog Out Functionality* for a complete Analog Out workflow.

SetAnalogOutMaster	<i>Set the Analog Out master channel</i>
--------------------	--

Description

Set the state-machine master for a selected Analog Out channel.

Usage

```
SetAnalogOutMaster(device, channel, master_channel)
```

Arguments

device	A "dwf4r_device" object returned by OpenDevice() that refers to a device connection that is still open.
channel	An integer scalar representing the zero-based Analog Out channel index.
master_channel	An integer scalar representing the zero-based Analog Out channel index to use as the state-machine master.

Details

A channel whose master differs from itself is controlled by the state machine of the master channel. Trigger, wait, run, and repeat settings therefore normally need to be configured only for the master channel.

Setting master_channel equal to channel appears to correspond to independent channel operation. This behavior is not explicitly described in the WaveForms SDK reference manual, but was observed experimentally with Analog Discovery 2.

This function updates the pending Analog Out configuration. Because automatic configuration is disabled by **dwf4r**, call ApplyAnalogOutSettings() to apply the setting without changing the current channel state, or StartAnalogOut() to apply the setting.

When starting synchronized channels, initialize the slave channel before starting the master channel.

Value

Invisibly returns NULL.

See Also

See the vignette *Basic Analog Out Functionality* for a complete Analog Out workflow.

SetAnalogOutRepeatTrigger

Set Analog Out repeat triggering

Description

Set whether a new trigger is included in each repeat cycle of a selected Analog Out channel.

Usage

```
SetAnalogOutRepeatTrigger(device, channel, repeat_trigger)
```

Arguments

device	A "dwf4r_device" object returned by <code>OpenDevice()</code> that refers to a device connection that is still open.
channel	An integer scalar representing the zero-based Analog Out channel index.
repeat_trigger	A logical scalar. If TRUE, a new trigger is included in each wait-run repeat cycle. If FALSE, repeated cycles proceed without waiting for another trigger.

Details

This setting controls trigger behavior during repetition and does not specify the number of repetitions. Use `SetAnalogOutRepeat()` to set the repeat count.

This function updates the pending Analog Out configuration. Because automatic configuration is disabled by **dwf4r**, call `ApplyAnalogOutSettings()` to apply the setting without changing the current channel state, or `StartAnalogOut()` to apply the setting and start generation.

Value

Invisibly returns NULL.

SetAnalogOutTiming *Set Analog Out timing*

Description

Set the run time, post-trigger wait time, or repeat count for a selected Analog Out channel.

SetAnalogOutRun() sets the run length of an Analog Out channel.

SetAnalogOutWait() sets the delay between receiving a trigger and beginning signal generation.

SetAnalogOutRepeat() sets the repeat count of an Analog Out channel.

SetAnalogOutTiming() sets one or more timing parameters of an Analog Out channel.

Usage

```
SetAnalogOutRun(device, channel, run_time)
```

```
SetAnalogOutWait(device, channel, wait_time)
```

```
SetAnalogOutRepeat(device, channel, repeat_count)
```

```
SetAnalogOutTiming(
    device,
    channel,
    run_time = NULL,
    wait_time = NULL,
    repeat_count = NULL
)
```

Arguments

device	A "dwf4r_device" object returned by OpenDevice() that refers to a device connection that is still open.
channel	An integer scalar representing the zero-based Analog Out channel index.
run_time	A numeric scalar specifying the run length in seconds. A value of 0 requests continuous generation.
wait_time	A numeric scalar specifying the delay, in seconds, between receiving a trigger and beginning signal generation. A value of 0 disables this delay.
repeat_count	An integer scalar specifying the repeat count. A value of 0 requests infinite repetition.

Details

These functions update the pending Analog Out configuration. Because automatic configuration is disabled by **dwf4r**, call ApplyAnalogOutSettings() to apply the settings without changing the current channel state, or StartAnalogOut() to apply the settings and start generation.

In SetAnalogOutTiming(), a NULL argument leaves the corresponding setting unchanged. Selected parameters are updated in the order run_time, wait_time, and repeat_count. If an error occurs, parameters set before the error remain in the pending configuration.

Value

Invisibly returns NULL.

See Also

See the vignette *Basic Analog Out Functionality* for a complete Analog Out workflow.

SetAnalogOutTriggerSlope

Set the Analog Out trigger slope

Description

Set the trigger slope for a selected Analog Out channel.

Usage

```
SetAnalogOutTriggerSlope(device, channel, slope)
```

Arguments

device	A "dwf4r_device" object returned by <code>OpenDevice()</code> that refers to a device connection that is still open.
channel	An integer scalar representing the zero-based Analog Out channel index.
slope	A string specifying the trigger slope. Recognized values are "rising", "falling", and "either".

Details

This function updates the pending Analog Out configuration. Because automatic configuration is disabled by **dwf4r**, call `ApplyAnalogOutSettings()` to apply the setting without changing the current channel state, or `StartAnalogOut()` to apply the setting and start generation.

Value

Invisibly returns NULL.

SetAnalogOutTriggerSource

Set the Analog Out trigger source

Description

Set the trigger source for a selected Analog Out channel.

Usage

```
SetAnalogOutTriggerSource(device, channel, source)
```

Arguments

device	A "dwf4r_device" object returned by OpenDevice() that refers to a device connection that is still open.
channel	An integer scalar representing the zero-based Analog Out channel index.
source	A string specifying the trigger source. Recognized values include "none", "pc", "detector_analog_in", "detector_digital_in", "analog_in", "digital_in", "digital_out", "analog_out_1", "analog_out_2", "external_1", and "external_2". This list is not exhaustive; available sources depend on the selected device and can be queried with GetDeviceTriggerSources().

Details

This function updates the pending Analog Out configuration. Because automatic configuration is disabled by **dwf4r**, call ApplyAnalogOutSettings() to apply the setting without changing the current channel state, or StartAnalogOut() to apply the setting and start generation.

Value

Invisibly returns NULL.

SetCarrier

Set carrier waveform parameters

Description

Set parameters of the carrier node for an Analog Out channel.

SetCarrierFunction() sets the waveform function of the carrier node.

SetCarrierFrequency() sets the frequency of the carrier node.

SetCarrierAmplitude() sets the amplitude of the carrier node.

SetCarrierOffset() sets the offset of the carrier node.

SetCarrierSymmetry() sets the symmetry of the carrier node.

SetCarrierPhase() sets the phase (in degrees) of the carrier node.

SetCarrier() sets one or more parameters of the carrier node for an Analog Out channel. Carrier parameters are updated sequentially. If an error occurs, parameters set before the error remain in the pending configuration but are not applied until ApplyAnalogOutSettings() or StartAnalogOut() is called.

Usage

```

SetCarrierFunction(device, channel, func)

SetCarrierFrequency(device, channel, frequency)

SetCarrierAmplitude(device, channel, amplitude)

SetCarrierOffset(device, channel, offset)

SetCarrierSymmetry(device, channel, symmetry)

SetCarrierPhase(device, channel, phase)

SetCarrier(
    device,
    channel,
    func = NULL,
    frequency = NULL,
    amplitude = NULL,
    offset = NULL,
    symmetry = NULL,
    phase = NULL
)

```

Arguments

device	A "dwf4r_device" object returned by OpenDevice() that refers to a device connection that is still open.
channel	An integer scalar representing the zero-based Analog Out channel index.
func	A string specifying the carrier waveform function. Available functions depend on the selected device and can be queried with GetAnalogOutNodeFunctionTypes(). The list of functions from the manual: • "dc" - DC value set as offset; • "sine" - sine waveform; • "square" - square waveform (offset +/- amplitude); • "triangle" - triangle waveform; • "ramp_up" - waveform with a ramp-up voltage at the beginning; • "ramp_down" - a waveform with a ramp-down voltage at the end; • "noise" - noise waveform from random samples; • "pulse" - pulse waveform (offset + amplitude); • "trapezium" - trapezium; • "sine_power" - sine with symmetry used as power function; • "sine_na" - sine waveform for Network Analyzer, dynamic frequency adjustment without glitch • "dual_custom" - double buffering of the waveform in the device for smooth switching. • "dual_pattern" - double buffering of the waveform in the device for smooth switching. • "custom_pattern" - waveform from custom repeated data with fixed sample rate;

	• "play_pattern" - waveform in stream play style (it provides constant sample rate); • "custom" - waveform from custom samples (optimizes for average requested frequency, sample output lengths may vary by one system frequency period); • "play" - waveform in stream play style (optimizes for average requested frequency).
frequency	A numeric scalar specifying the frequency in Hz.
amplitude	A numeric scalar specifying the carrier amplitude, in volts. For symmetric waveforms such as a sine wave, this is the voltage excursion from the configured offset to either peak. The output therefore ranges from offset - amplitude to offset + amplitude, corresponding to a peak-to-peak voltage of 2 * amplitude.
offset	A numeric scalar specifying the carrier offset, in volts. For symmetric waveforms, the offset is the midpoint between the minimum and maximum output voltages.
symmetry	A numeric scalar specifying the symmetry in percent.
phase	A numeric scalar specifying the carrier phase in degrees.

Details

These functions set carrier parameters only. Use ApplyAnalogOutSettings() to send updated settings to the device without changing the current channel state, or StartAnalogOut() to apply settings and start generation.

The requested value may differ from the value actually configured by the device; use the corresponding GetCarrier*() function to retrieve the actual value.

Value

Invisibly returns NULL.

See Also

See the vignette *Basic Analog Out Functionality* for a complete Analog Out workflow.

SetCarrierData	<i>Set carrier custom waveform samples</i>
----------------	--

Description

Upload a custom sample vector for the carrier node of an Analog Out channel. Samples must be normalized to the range [-1, 1].

This function uploads sample data but does not select the custom waveform function or start generation. Use SetCarrierFunction(..., func = "custom") and then call ApplyAnalogOutSettings() or StartAnalogOut().

Usage

SetCarrierData(device, channel, data)

Arguments

device	A "dwf4r_device" object returned by OpenDevice() that refers to a device connection that is still open.
channel	An integer scalar representing the zero-based Analog Out channel index.
data	A numeric vector of waveform samples normalized to the range [-1, 1].

Value

Invisibly returns NULL.

See Also

See the vignette *Basic Analog Out Functionality* for a complete Analog Out workflow.

SetDigitalOutChannelSettings

Set Digital Out channel settings

Description

Set the output mode, signal-generation function, or idle output mode of a selected Digital Out channel.

SetDigitalOutOutput() sets the output mode of a Digital Out channel.

SetDigitalOutFunction() sets the signal-generation function of a Digital Out channel.

SetDigitalOutIdle() sets the idle output mode of a Digital Out channel.

Usage

```
SetDigitalOutOutput(device, channel, output, .validate_digital_out = TRUE)
```

```
SetDigitalOutFunction(device, channel, func, .validate_digital_out = TRUE)
```

```
SetDigitalOutIdle(device, channel, idle, .validate_digital_out = TRUE)
```

Arguments

device	A "dwf4r_device" object returned by OpenDevice() that refers to a device connection that is still open.
channel	An integer scalar representing the zero-based Digital Out channel index.
output	A string specifying the output mode. Possible values are "push_pull", "open_drain", "open_source", and "three_state". Supported values for a selected channel can be queried with GetDigitalOutOutputModes().
.validate_digital_out	A logical scalar indicating whether AssertDigitalOut() should be called. This argument is intended for advanced use to avoid repeated validation in controlled sequences of operations. Set to FALSE only when the device and relevant Digital Out context, such as the channel and supported configuration choices, have already been validated. The caller is responsible for ensuring that the device remains open and the validated context remains valid. Other argument validation and SDK error handling remain active.

func	A string specifying the signal-generation function. Possible values are "pulse", "custom", "random", "rom", "state", and "play". Supported values for a selected channel can be queried with <code>GetDigitalOutFunctionTypes()</code> .
idle	A string specifying the idle output mode. Possible values are "initial", "low", "high", and "three_state". Supported values for a selected channel can be queried with <code>GetDigitalOutIdleModes()</code> .

Details

The WaveForms SDK and the public **dwf4r** API use different terms for selecting the type of generated signal. The SDK uses *type*, whereas the R API uses *function*. The latter is used primarily to maintain consistent naming between Analog Out and Digital Out functionality.

Each requested value is checked against the capabilities reported for the selected channel. Relationships between the output mode, function, and idle mode are not validated; a combination of individually supported values is therefore not necessarily a valid Digital Out configuration.

These functions update the pending Digital Out configuration. With automatic configuration disabled by **dwf4r**, changing a setting does not stop current Digital Out generation. Call `StartDigitalOut()` to start or restart the instrument with the updated configuration. To stop generation, call `StopDigitalOut()` explicitly.

Value

Invisibly returns NULL.

See Also

See the vignette *Basic Digital Out Functionality* for a complete Digital Out workflow.

SetDigitalOutCounter *Set Digital Out low and high counters*

Description

Set the low and high counter values of a selected Digital Out channel.

Usage

```
SetDigitalOutCounter(
  device,
  channel,
  low_count,
  high_count,
  .validate_digital_out = TRUE
)
```

Arguments

device	A "dwf4r_device" object returned by OpenDevice() that refers to a device connection that is still open.
channel	An integer scalar representing the zero-based Digital Out channel index.
low_count, high_count	Non-negative whole-number scalars specifying the low and high counter values respectively. The supported range can be queried with GetDigitalOutCounterRange().
.validate_digital_out	A logical scalar indicating whether AssertDigitalOut() should be called. This argument is intended for advanced use to avoid repeated validation in controlled sequences of operations. Set to FALSE only when the device and relevant Digital Out context, such as the channel and supported configuration choices, have already been validated. The caller is responsible for ensuring that the device remains open and the validated context remains valid. Other argument validation and SDK error handling remain active.

Details

For pulse generation, the low and high counters determine how many counts the output remains low and high, respectively. The internal clock frequency and divider value determine the duration of one count.

The WaveForms SDK specifies that a zero low or high counter value prevents the output level from toggling.

This function updates the pending Digital Out configuration. With automatic configuration disabled by **dwf4r**, changing the counter values does not stop current Digital Out generation. Call StartDigitalOut() to start or restart the instrument with the updated configuration. To stop generation, call StopDigitalOut() explicitly.

Value

Invisibly returns NULL.

See Also

See the vignette *Basic Digital Out Functionality* for a complete Digital Out workflow.

SetDigitalOutCustomPattern

Set a custom Digital Out pattern

Description

Set the custom pattern of a selected Digital Out channel.

Usage

```
SetDigitalOutCustomPattern(
    device,
    channel,
    pattern,
    is_three_state = FALSE,
    .validate_digital_out = TRUE
)
```

Arguments

device	A "dwf4r_device" object returned by <code>OpenDevice()</code> that refers to a device connection that is still open.
channel	An integer scalar representing the zero-based Digital Out channel index.
pattern	A non-empty numeric or character vector specifying the custom pattern. A numeric pattern must contain only 0 and 1. A character pattern must contain only "l", "h", and "z", which represent logical low, logical high, and high-impedance samples, respectively. The "z" value can only be used when <code>is_three_state = TRUE</code> .
is_three_state	A logical scalar indicating whether the pattern should be encoded for three-state output. This argument controls only pattern encoding; it does not configure the Digital Out output mode.
.validate_digital_out	A logical scalar indicating whether <code>AssertDigitalOut()</code> should be called. This argument is intended for advanced use to avoid repeated validation in controlled sequences of operations. Set to FALSE only when the device and relevant Digital Out context, such as the channel and supported configuration choices, have already been validated. The caller is responsible for ensuring that the device remains open and the validated context remains valid. Other argument validation and SDK error handling remain active.

Details

This function uploads custom data but does not change the signal-generation function. Before starting generation, configure the channel with `SetDigitalOutFunction(..., func = "custom")`.

With `is_three_state = FALSE`, each pattern element consumes one custom-data bit. With `is_three_state = TRUE`, each pattern element consumes two custom-data bits: an output-value (IO) bit followed by an output-enable (OE) bit. For "l" and "h", the output is enabled; for "z", it is disabled.

When `is_three_state = TRUE`, configure the corresponding channel output mode separately with `SetDigitalOutOutput(..., output = "three_state")`.

The maximum number of custom-data bits depends on the selected channel and can be queried with `GetDigitalOutMaxDataBits()`. Three-state patterns consume two custom-data bits per pattern element.

Setting a custom pattern automatically configures the Digital Out counters used internally to step through the pattern. These counters normally do not need to be configured manually for custom-pattern generation. Changing them with `SetDigitalOutCounter()` or `SetDigitalOutInitialCounter()` after setting the pattern may change how the pattern is generated.

This function updates the Digital Out configuration without stopping current generation. Call `StartDigitalOut()` to start or restart the instrument with the updated custom pattern. To stop generation, call `StopDigitalOut()` explicitly.

Value

Invisibly returns NULL.

See Also

See the vignette *Basic Digital Out Functionality* for a complete Digital Out workflow.

SetDigitalOutCustomSampleRate

Set the Digital Out custom-pattern sample rate

Description

Set the sample rate used for custom-pattern generation on a selected Digital Out channel.

Usage

```
SetDigitalOutCustomSampleRate(
  device,
  channel,
  sample_rate,
  .validate_digital_out = TRUE
)
```

Arguments

device	A "dwf4r_device" object returned by OpenDevice() that refers to a device connection that is still open.
channel	An integer scalar representing the zero-based Digital Out channel index.
sample_rate	A positive numeric scalar specifying the custom-pattern sample rate in Hz. The sample rate is the rate at which individual pattern elements are generated, not the repetition frequency of the complete pattern. For example, at a sample rate of 1 kHz, each pattern element lasts 1 ms.
.validate_digital_out	A logical scalar indicating whether AssertDigitalOut() should be called. This argument is intended for advanced use to avoid repeated validation in controlled sequences of operations. Set to FALSE only when the device and relevant Digital Out context, such as the channel and supported configuration choices, have already been validated. The caller is responsible for ensuring that the device remains open and the validated context remains valid. Other argument validation and SDK error handling remain active.

Details

The custom-pattern sample rate is determined by the Digital Out internal clock frequency divided by the ordinary channel divider. Because the divider is a whole number, the requested sample rate is converted to the nearest supported divider value. The resulting sample rate may therefore differ slightly from the requested value.

The initial timing interval may differ from subsequent intervals when the initial divider differs from the ordinary channel divider.

This function changes the ordinary divider but does not modify the initial divider. Use `GetDigitalOutCustomSampleRate()` to retrieve the sample rate corresponding to the configured divider.

This function updates the pending Digital Out configuration. With automatic configuration disabled by **dwf4r**, changing the sample rate does not stop current Digital Out generation. Call `StartDigitalOut()` to start or restart the instrument with the updated configuration. To stop generation, call `StopDigitalOut()` explicitly.

Value

Invisibly returns `NULL`.

See Also

See the vignette *Basic Digital Out Functionality* for a complete Digital Out workflow.

SetDigitalOutDivider *Set the Digital Out divider*

Description

Set the divider used by a selected Digital Out channel after the initial divider expires. The internal clock frequency and divider value determine the duration of one counter step.

Usage

```
SetDigitalOutDivider(device, channel, divider, .validate_digital_out = TRUE)
```

Arguments

device	A "dwf4r_device" object returned by <code>OpenDevice()</code> that refers to a device connection that is still open.
channel	An integer scalar representing the zero-based Digital Out channel index.
divider	A non-negative whole-number scalar specifying the divider value. The supported range depends on the selected channel and can be queried with <code>GetDigitalOutDividerRange()</code> .
.validate_digital_out	A logical scalar indicating whether <code>AssertDigitalOut()</code> should be called. This argument is intended for advanced use to avoid repeated validation in controlled sequences of operations. Set to <code>FALSE</code> only when the device and relevant Digital Out context, such as the channel and supported configuration choices, have already been validated. The caller is responsible for ensuring that the device remains open and the validated context remains valid. Other argument validation and SDK error handling remain active.

Details

This function updates the pending Digital Out configuration. With automatic configuration disabled by **dwf4r**, changing the divider does not stop current Digital Out generation. Call `StartDigitalOut()` to start or restart the instrument with the updated configuration. To stop generation, call `StopDigitalOut()` explicitly.

Value

Invisibly returns NULL.

See Also

See the vignette *Basic Digital Out Functionality* for a complete Digital Out workflow.

SetDigitalOutInitialCounter

Set the initial Digital Out counter

Description

Set the initial output state and counter value of a selected Digital Out channel.

Usage

```
SetDigitalOutInitialCounter(
  device,
  channel,
  initial_state,
  initial_count,
  .validate_digital_out = TRUE
)
```

Arguments

device	A "dwf4r_device" object returned by OpenDevice() that refers to a device connection that is still open.
channel	An integer scalar representing the zero-based Digital Out channel index.
initial_state	A string specifying the initial output state, either "low" or "high".
initial_count	A non-negative whole-number scalar specifying the initial counter value. The supported range can be queried with GetDigitalOutCounterRange(). An initial count of zero is accepted.
.validate_digital_out	A logical scalar indicating whether AssertDigitalOut() should be called. This argument is intended for advanced use to avoid repeated validation in controlled sequences of operations. Set to FALSE only when the device and relevant Digital Out context, such as the channel and supported configuration choices, have already been validated. The caller is responsible for ensuring that the device remains open and the validated context remains valid. Other argument validation and SDK error handling remain active.

Details

The initial state and counter value are loaded when the Digital Out instrument enters the Running state. The initial counter determines the number of counts before the ordinary low and high counter sequence begins. The internal clock frequency and divider values determine the duration of each count.

This function updates the pending Digital Out configuration. With automatic configuration disabled by **dwf4r**, changing the initial counter does not stop current Digital Out generation. Call `StartDigitalOut()` to start or restart the instrument with the updated configuration. To stop generation, call `StopDigitalOut()` explicitly.

Value

Invisibly returns NULL.

See Also

See the vignette *Basic Digital Out Functionality* for a complete Digital Out workflow.

SetDigitalOutInitialDivider

Set the initial Digital Out divider

Description

Set the divider value initially loaded when the Digital Out instrument enters the Running state. The internal clock frequency and initial divider value determine the duration of the first counter step.

Usage

```
SetDigitalOutInitialDivider(
  device,
  channel,
  initial_divider,
  .validate_digital_out = TRUE
)
```

Arguments

device A "dwf4r_device" object returned by `OpenDevice()` that refers to a device connection that is still open.

channel An integer scalar representing the zero-based Digital Out channel index.

initial_divider A non-negative whole-number scalar specifying the initial divider value. The supported range depends on the selected channel and can be queried with `GetDigitalOutDividerRange()`.

.validate_digital_out

A logical scalar indicating whether `AssertDigitalOut()` should be called. This argument is intended for advanced use to avoid repeated validation in controlled sequences of operations. Set to FALSE only when the device and relevant Digital Out context, such as the channel and supported configuration choices, have already been validated. The caller is responsible for ensuring that the device remains open and the validated context remains valid. Other argument validation and SDK error handling remain active.

Details

This function updates the pending Digital Out configuration. With automatic configuration disabled by **dwf4r**, changing the initial divider does not stop current Digital Out generation. Call `StartDigitalOut()` to start or restart the instrument with the updated configuration. To stop generation, call `StopDigitalOut()` explicitly.

Value

Invisibly returns NULL.

See Also

See the vignette *Basic Digital Out Functionality* for a complete Digital Out workflow.

SetDigitalOutRepeatTrigger

Set Digital Out repeat triggering

Description

Set whether a trigger is included in each wait-run repeat cycle of the Digital Out instrument. This setting is shared by all Digital Out channels.

Usage

```
SetDigitalOutRepeatTrigger(device, repeat_trigger, .validate_device = TRUE)
```

Arguments

- | | |
|------------------|--|
| device | A "dwf4r_device" object returned by <code>OpenDevice()</code> that refers to a device connection that is still open. |
| repeat_trigger | A logical scalar. If TRUE, each subsequent repeat cycle returns to the Armed state before proceeding through Wait and Running. If FALSE, subsequent cycles proceed without waiting for another trigger. The SDK default is FALSE. |
| .validate_device | A logical scalar indicating whether <code>AssertDevice()</code> should be called. This argument is intended for advanced use to avoid repeated validation in controlled sequences of operations. Set to FALSE only when the device has already been validated and is known to remain open. Other argument validation and SDK error handling remain active. |

Details

This setting controls trigger behavior during repetition and does not specify the number of repetitions. Use `SetDigitalOutRepeat()` to set the repeat count. The selected trigger source determines the trigger condition; enabling repeat triggering does not itself select a source.

A run time of 0 requests continuous generation, so the instrument does not reach another wait-run repeat cycle.

This function updates the pending Digital Out configuration. Because automatic configuration is disabled by **dwf4r**, call `StartDigitalOut()` to start or restart the instrument with the updated setting. To stop generation, call `StopDigitalOut()`.

Value

Invisibly returns NULL.

SetDigitalOutTiming	<i>Set Digital Out timing</i>
---------------------	-------------------------------

Description

Set the run time, post-trigger wait time, or repeat count for the Digital Out instrument. These settings are shared by all Digital Out channels.

SetDigitalOutRun() sets the run length of the Digital Out instrument.

SetDigitalOutWait() sets the delay between receiving a trigger and beginning signal generation.

SetDigitalOutRepeat() sets the repeat count of the Digital Out instrument.

SetDigitalOutTiming() sets one or more timing parameters of the Digital Out instrument.

Usage

```
SetDigitalOutRun(device, run_time, .validate_device = TRUE)
```

```
SetDigitalOutWait(device, wait_time, .validate_device = TRUE)
```

```
SetDigitalOutRepeat(device, repeat_count, .validate_device = TRUE)
```

```
SetDigitalOutTiming(
  device,
  run_time = NULL,
  wait_time = NULL,
  repeat_count = NULL,
  .validate_device = TRUE
)
```

Arguments

device	A "dwf4r_device" object returned by OpenDevice() that refers to a device connection that is still open.
run_time	A finite, non-negative numeric scalar specifying the run length in seconds. A value of 0 requests continuous generation.
.validate_device	A logical scalar indicating whether AssertDevice() should be called. This argument is intended for advanced use to avoid repeated validation in controlled sequences of operations. Set to FALSE only when the device has already been validated and is known to remain open. Other argument validation and SDK error handling remain active.
wait_time	A finite, non-negative numeric scalar specifying the delay, in seconds, between receiving a trigger and beginning signal generation. A value of 0 disables this delay.
repeat_count	A non-negative whole-number numeric scalar specifying the number of wait-run cycles. A value of 0 requests infinite repetition.

Details

Nonzero values are checked against the ranges reported by `GetDigitalOutRunRange()`, `GetDigitalOutWaitRange()`, and `GetDigitalOutRepeatRange()`. Zero is accepted as a special value even when the corresponding reported range has a positive lower bound.

The repeat count controls the number of wait-run cycles, not the number of individual pulses or copies of a custom pattern. Whether each cycle requires another trigger depends on the SDK repeat-trigger setting, which is disabled by default. These functions do not change that setting. With a run time of 0, generation continues indefinitely and the repeat count does not impose a finite stopping point.

These functions update the pending Digital Out configuration. Because automatic configuration is disabled by **dwf4r**, call `StartDigitalOut()` to start or restart the instrument with the updated settings.

In `SetDigitalOutTiming()`, a NULL argument leaves the corresponding setting unchanged. At least one timing parameter must be supplied. Selected parameters are validated and set in the order `run_time`, `wait_time`, and `repeat_count`. If an error occurs, parameters set before the error remain in the pending configuration.

Value

Invisibly returns NULL.

See Also

See the vignette *Basic Digital Out Functionality* for a complete Digital Out workflow.

SetDigitalOutTriggerSlope

Set the Digital Out trigger slope

Description

Set the trigger slope for the Digital Out instrument. This setting is shared by all Digital Out channels.

Usage

```
SetDigitalOutTriggerSlope(device, slope, .validate_device = TRUE)
```

Arguments

<code>device</code>	A "dwf4r_device" object returned by <code>OpenDevice()</code> that refers to a device connection that is still open.
<code>slope</code>	A string specifying the trigger slope. Recognized values are "rising", "falling", and "either". Supported values can be queried with <code>GetDeviceTriggerSlopes()</code> .
<code>.validate_device</code>	A logical scalar indicating whether <code>AssertDevice()</code> should be called. This argument is intended for advanced use to avoid repeated validation in controlled sequences of operations. Set to FALSE only when the device has already been validated and is known to remain open. Other argument validation and SDK error handling remain active.

Details

This function updates the pending Digital Out configuration. Because automatic configuration is disabled by **dwf4r**, call `StartDigitalOut()` to start or restart the instrument with the updated setting. To stop generation, call `StopDigitalOut()`.

Value

Invisibly returns NULL.

SetDigitalOutTriggerSource

Set the Digital Out trigger source

Description

Set the trigger source for the Digital Out instrument. This setting is shared by all Digital Out channels.

Usage

```
SetDigitalOutTriggerSource(device, source, .validate_device = TRUE)
```

Arguments

device	A "dwf4r_device" object returned by <code>OpenDevice()</code> that refers to a device connection that is still open.
source	A string specifying the trigger source. Available sources depend on the selected device and can be queried with <code>GetDeviceTriggerSources()</code> . Names use the same convention as <code>SetAnalogOutTriggerSource()</code> . The SDK default is "none".
.validate_device	A logical scalar indicating whether <code>AssertDevice()</code> should be called. This argument is intended for advanced use to avoid repeated validation in controlled sequences of operations. Set to FALSE only when the device has already been validated and is known to remain open. Other argument validation and SDK error handling remain active.

Details

This function updates the pending Digital Out configuration. Because automatic configuration is disabled by **dwf4r**, call `StartDigitalOut()` to start or restart the instrument with the updated setting. To stop generation, call `StopDigitalOut()`.

For PC triggering, select `source = "pc"`, start the instrument with `StartDigitalOut()`, and use `TriggerDevice()` to issue a PC trigger pulse while the instrument is armed. Other instruments configured to use the PC trigger can also respond to this pulse.

Value

Invisibly returns NULL.

TriggerDevice	<i>Trigger a WaveForms device</i>
---------------	-----------------------------------

Description

Generate a PC trigger pulse for a WaveForms device.

Instruments configured to use the "pc" trigger source can respond to this pulse.

Usage

```
TriggerDevice(device)
```

Arguments

device	A "dwf4r_device" object returned by OpenDevice() that refers to a device connection that is still open.
--------	---

Value

Invisibly returns NULL.

Index

ApplyAnalogOutSettings
(ConfigureAnalogOut), 5

CarrierEnableDisable, 3

CloseAllDevices, 3

CloseDevice, 4

ConfigureAnalogOut, 5

ConfigureDigitalOut, 5

DigitalOutChannelEnableDisable, 6

DisableCarrier (CarrierEnableDisable), 3

DisableDigitalOutChannel
(DigitalOutChannelEnableDisable),
6

EnableCarrier (CarrierEnableDisable), 3

EnableDigitalOutChannel
(DigitalOutChannelEnableDisable),
6

GetAnalogOutChannelCount
(GetAnalogOutInfo), 8

GetAnalogOutChannelNodes
(GetAnalogOutInfo), 8

GetAnalogOutIdle, 7

GetAnalogOutIdleModes
(GetAnalogOutInfo), 8

GetAnalogOutInfo, 8

GetAnalogOutMaster, 9

GetAnalogOutNodeAmplitudeRange
(GetAnalogOutInfo), 8

GetAnalogOutNodeFrequencyRange
(GetAnalogOutInfo), 8

GetAnalogOutNodeFunctionTypes
(GetAnalogOutInfo), 8

GetAnalogOutNodeOffsetRange
(GetAnalogOutInfo), 8

GetAnalogOutNodePhaseRange
(GetAnalogOutInfo), 8

GetAnalogOutNodeSampleCountRange
(GetAnalogOutInfo), 8

GetAnalogOutNodeSymmetryRange
(GetAnalogOutInfo), 8

GetAnalogOutRemaining, 10

GetAnalogOutRemainingRepeat
(GetAnalogOutRemaining), 10

GetAnalogOutRemainingRun
(GetAnalogOutRemaining), 10

GetAnalogOutRepeat
(GetAnalogOutTiming), 12

GetAnalogOutRepeatRange
(GetAnalogOutInfo), 8

GetAnalogOutRepeatTrigger, 11

GetAnalogOutRun (GetAnalogOutTiming), 12

GetAnalogOutRunRange
(GetAnalogOutInfo), 8

GetAnalogOutStatus, 11

GetAnalogOutTiming, 12

GetAnalogOutTriggerSlope, 13

GetAnalogOutTriggerSource, 13

GetAnalogOutWait (GetAnalogOutTiming),
12

GetAnalogOutWaitRange
(GetAnalogOutInfo), 8

GetCarrierAmplitude (GetCarrierValues),
15

GetCarrierFrequency (GetCarrierValues),
15

GetCarrierFunction, 14

GetCarrierOffset (GetCarrierValues), 15

GetCarrierPhase (GetCarrierValues), 15

GetCarrierSettings, 14

GetCarrierSymmetry (GetCarrierValues),
15

GetCarrierValues, 15

GetDeviceTriggerInfo, 16

GetDeviceTriggerSlopes
(GetDeviceTriggerInfo), 16

GetDeviceTriggerSources
(GetDeviceTriggerInfo), 16

GetDigitalOutChannelCount
(GetDigitalOutInfo), 22

GetDigitalOutChannelInfo, 17

GetDigitalOutChannelSettings, 18

GetDigitalOutCounter, 19

GetDigitalOutCounterRange
(GetDigitalOutChannelInfo), 17

- GetDigitalOutCustomSampleRate, 20
- GetDigitalOutDivider
 - (GetDigitalOutDividerValues), 21
- GetDigitalOutDividerRange
 - (GetDigitalOutChannelInfo), 17
- GetDigitalOutDividerValues, 21
- GetDigitalOutFunction
 - (GetDigitalOutChannelSettings), 18
- GetDigitalOutFunctionTypes
 - (GetDigitalOutChannelInfo), 17
- GetDigitalOutIdle
 - (GetDigitalOutChannelSettings), 18
- GetDigitalOutIdleModes
 - (GetDigitalOutChannelInfo), 17
- GetDigitalOutInfo, 22
- GetDigitalOutInitialCounter, 23
- GetDigitalOutInitialDivider
 - (GetDigitalOutDividerValues), 21
- GetDigitalOutInternalClockFrequency
 - (GetDigitalOutInfo), 22
- GetDigitalOutMaxDataBits
 - (GetDigitalOutChannelInfo), 17
- GetDigitalOutOutput
 - (GetDigitalOutChannelSettings), 18
- GetDigitalOutOutputModes
 - (GetDigitalOutChannelInfo), 17
- GetDigitalOutRemaining, 24
- GetDigitalOutRemainingRepeat
 - (GetDigitalOutRemaining), 24
- GetDigitalOutRemainingRun
 - (GetDigitalOutRemaining), 24
- GetDigitalOutRepeat
 - (GetDigitalOutTiming), 26
- GetDigitalOutRepeatRange
 - (GetDigitalOutInfo), 22
- GetDigitalOutRepeatTrigger, 25
- GetDigitalOutRun (GetDigitalOutTiming), 26
- GetDigitalOutRunRange
 - (GetDigitalOutInfo), 22
- GetDigitalOutStatus, 26
- GetDigitalOutTiming, 26
- GetDigitalOutTriggerSlope, 28
- GetDigitalOutTriggerSource, 28
- GetDigitalOutWait
 - (GetDigitalOutTiming), 26
- GetDigitalOutWaitRange
 - (GetDigitalOutInfo), 22
- IsCarrierEnabled, 29
- IsDigitalOutChannelEnabled, 29
- ListDevices, 30
- OpenDevice, 31
- ResetAnalogOut, 32
- ResetDigitalOut, 33
- SetAnalogOutIdle, 33
- SetAnalogOutMaster, 34
- SetAnalogOutRepeat
 - (SetAnalogOutTiming), 36
- SetAnalogOutRepeatTrigger, 35
- SetAnalogOutRun (SetAnalogOutTiming), 36
- SetAnalogOutTiming, 36
- SetAnalogOutTriggerSlope, 37
- SetAnalogOutTriggerSource, 38
- SetAnalogOutWait (SetAnalogOutTiming), 36
- SetCarrier, 38
- SetCarrierAmplitude (SetCarrier), 38
- SetCarrierData, 40
- SetCarrierFrequency (SetCarrier), 38
- SetCarrierFunction (SetCarrier), 38
- SetCarrierOffset (SetCarrier), 38
- SetCarrierPhase (SetCarrier), 38
- SetCarrierSymmetry (SetCarrier), 38
- SetDigitalOutChannelSettings, 41
- SetDigitalOutCounter, 42
- SetDigitalOutCustomPattern, 43
- SetDigitalOutCustomSampleRate, 45
- SetDigitalOutDivider, 46
- SetDigitalOutFunction
 - (SetDigitalOutChannelSettings), 41
- SetDigitalOutIdle
 - (SetDigitalOutChannelSettings), 41
- SetDigitalOutInitialCounter, 47
- SetDigitalOutInitialDivider, 48
- SetDigitalOutOutput
 - (SetDigitalOutChannelSettings), 41
- SetDigitalOutRepeat
 - (SetDigitalOutTiming), 50
- SetDigitalOutRepeatTrigger, 49
- SetDigitalOutRun (SetDigitalOutTiming), 50
- SetDigitalOutTiming, 50

SetDigitalOutTriggerSlope, [51](#)
SetDigitalOutTriggerSource, [52](#)
SetDigitalOutWait
 (SetDigitalOutTiming), [50](#)
StartAnalogOut (ConfigureAnalogOut), [5](#)
StartDigitalOut (ConfigureDigitalOut), [5](#)
StopAnalogOut (ConfigureAnalogOut), [5](#)
StopDigitalOut (ConfigureDigitalOut), [5](#)

TriggerDevice, [53](#)